three common ways of implementing an associative container like your lookup table:

- A binary tree, such as a red-black tree
- A sorted array
- A hash table

A binary tree doesn't provide much scope for extending the opportunities for concurrency; every lookup or modification has to start by accessing the root node, which therefore has to be locked. Although this lock can be released as the accessing thread moves down the tree, this isn't much better than a single lock across the whole data structure.

A sorted array is even worse, because you can't tell in advance where in the array a given data value is going to be, so you need a single lock for the whole array.

That leaves the hash table. Assuming a fixed number of buckets, which bucket a key belongs to is purely a property of the key and its hash function. This means you can safely have a separate lock per bucket. If you again use a mutex that supports multiple readers or a single writer, you increase the opportunities for concurrency *N*-fold, where *N* is the number of buckets. The downside is that you need a good hash function for the key. The C++ Standard Library provides the `std::hash<>` template, which you can use for this purpose. It's already specialized for the fundamental types such as `int` and common library types such as `std::string`, and the user can easily specialize it for other key types. If you follow the lead of the standard unordered containers and take the type of the function object to use for doing the hashing as a template parameter, the user can choose whether to specialize `std::hash<>` for their key type or provide a separate hash function.

So, let's look at some code. What might the implementation of a thread-safe lookup table look like? One possibility is shown here.

Listing 6.11. A thread-safe lookup table